

End-to-End DSL Compiler Project Report

Name: Rithanya M

Registration Number: RA2311026050117

GitHub Repository: (<https://github.com/rithanya06/Compiler-Design-DSL/tree/main>)

1. Architecture Flow

The designed DSL compiler follows a structured compilation pipeline to transform input source code into an intermediate representation.

Flow:

Source Code (input.txt)

↓

Lexical Analysis (Flex)

↓

Token Stream

↓

Syntax Analysis (Bison)

↓

Abstract Syntax Tree (AST Construction)

↓

Semantic Validation

↓

Intermediate Code Generation

↓

Final Output

This pipeline ensures that the input program is processed step-by-step, verifying correctness and generating meaningful output.

2. Modules Explanation

Lexical Analyzer (lexer.l)

The lexical analyzer scans the input program and breaks it into tokens such as identifiers, numbers, and operators. It uses pattern matching to recognize valid symbols.

Syntax Parser (parser.y)

The parser checks whether the input follows the defined grammar rules. It ensures that expressions and assignments are syntactically correct.

Abstract Syntax Tree (ast.c)

This module represents the structure of the program in a hierarchical format. It simplifies further processing by organizing expressions logically.

Semantic Analyzer (semantic.c)

The semantic phase verifies the correctness of variable usage and operations. It ensures that assignments and expressions are valid.

Intermediate Code Generator (codegen.c)

This module generates a simplified intermediate representation of the input program. It converts expressions into a form that can be easily interpreted or optimized.

3. Sample Execution & Output

Build Commands

```
bison -d parser.y
```

```
flex lexer.l
```

```
gcc lex.yy.c parser.tab.c ast.c semantic.c codegen.c -o compiler
```

```
./compiler < ../test/input.txt
```

Sample Input

```
x = 5
```

```
y = 10
```

```
z = x + y
```

Sample Output

[Lexer] Identifier token detected

[Lexer] Number token detected

[Parser] Assignment statement recognized

[Semantic] Valid variable assignment

[CodeGen] Intermediate code: $\text{temp} = x + y$

4. Conclusion

This project demonstrates a simplified yet complete compiler design workflow. Each stage of compilation—from lexical analysis to intermediate code generation—is implemented in a modular manner. The project provides a clear understanding of how programming languages are processed internally and highlights the importance of each compiler phase.